

Argus: A Typed Interface Layer for Public-Coin Arguments and Reductions

Ricardo Perello

Final project report, June 2026

Abstract

Argus is a Rust research prototype for describing public-coin interactive arguments and reductions as typed protocol objects. The project is motivated by a gap in the existing arkworks proof-system stack. Spongefish **already provides** Fiat–Shamir transcript machinery and non-interactive argument strings, but a protocol written directly against spongefish state is primarily an imperative transcript program: it calls transcript methods to record prover messages, derive verifier messages, and replay proof bytes. Such code is useful for building non-interactive proofs, but it is not a clean interface for the underlying interactive argument or reduction.

Argus introduces that missing layer. A protocol author writes only the public-coin message flow: the prover sends prover messages and reads public verifier messages; the verifier reads prover messages and sends public verifier messages. A hand-written protocol in this interface can run interactively or compile through `spongefish-dsfs` into a non-interactive argument or reduction. The same interface can also be the thing produced by a future compiler: for example, an IOP-to-iBCS compiler could output an IA, and that IA could then run through the same backends. The main design result is a typed interface that covers arguments and reductions, interactive and non-interactive execution, and plain and preprocessed protocols without collapsing all cases into one maximal object. Executable protocols are further factored into prover and verifier capabilities. Authors can still describe a complete conversation at once, while DSFS can compile a genuinely one-sided body into a prover-only or verifier-only non-interactive object. This provides the dependency boundary needed by future recursion and accumulation code, where an outer prover may need an inner verifier without the inner prover.

1 Motivation

Many modern proof systems are easiest to describe as public-coin interactive protocols. A prover sends algebraic data, commitments, or oracle-derived messages; the verifier samples public randomness; and the protocol repeats until the verifier either accepts or computes a reduced target claim. In a non-interactive proof system, the same public randomness is produced by a Fiat–Shamir transcript. The implementation must therefore serialize prover messages, absorb them into a transcript, derive verifier challenges, and replay the same operations during verification.

Spongefish gives the arkworks ecosystem this transcript machinery. For example, the spongefish Schnorr example is written around prover and verifier transcript state: the prover records prover messages and asks the transcript for verifier messages, while the verifier reads messages from a non-interactive argument string, absorbs them, derives challenges, and checks end-of-file. This is the right level of abstraction for a transcript library. It is not the right level of abstraction for describing an interactive argument as a reusable protocol. The protocol and the compiled transcript are entangled in the same imperative code.

Argus exists to separate these two concerns. The protocol-facing interface says only what the interactive protocol does: which messages the prover sends, which public messages the verifier sends, and what the verifier outputs. Transcript mechanics are delegated to a backend. This means the same protocol object can be used as an input to DSFS, as a live interactive protocol, or as a target interface for a future compiler that turns a richer formalism into an interactive argument.

This last point matters for the broader arkworks post-quantum direction. The planned `arkworks-pq` workspace is intended to provide trait surfaces and backends for vector commitments, Merkle trees, FRI/STIR/WHIR-style components, STARK ingredients, linear codes, binary-field primitives, and related hash-based SNARK infrastructure. Those systems need a clean argument layer. Long term, the desired shape is not merely

$$IA \xrightarrow{DSFS} NARG,$$

but also

$$IOP \xrightarrow{iBCS} IA \xrightarrow{DSFS} NARG.$$

In this pipeline, the IA interface is simultaneously a target language, a source language, and an executable object.

2 Design Problem

The core design problem is a small matrix of protocol forms. Argus wants to represent:

Axis	Cases
Verifier output	argument / reduction
Execution mode	interactive / non-interactive
Setup model	plain / preprocessed

These three binary choices give eight useful combinations. A plain interactive argument is the familiar accept/reject public-coin protocol. An interactive reduction outputs a target instance instead of a Boolean. A non-interactive argument or reduction is the DSFS-compiled form. A preprocessed protocol has an index, prover key, verifier key, and a committed representation of the indexed material that must be fixed before the first challenge.

One possible design is top-down: define one maximally general object, roughly a preprocessing interactive reduction, and recover all simpler objects by degeneration. A plain protocol has a trivial index. An argument is a reduction whose target is a unit type or whose verifier output is interpreted as accept/reject. Non-preprocessed protocols have dummy keys. This is compact, but it weakens the type meaning of the API. A plain interactive argument would appear to have preprocessing methods even when preprocessing is not part of the protocol.

Argus takes the opposite approach. It starts with a bottom-up capability tree. All protocol objects share protocol identity. Arguments add instance and witness types. Reductions add source and target relation types. Preprocessing adds an index, prover key, verifier key, and preprocessing function. Executable interactive protocols add channel methods, with prover and verifier execution represented as separate capabilities. Non-interactive protocols add session-bound proving and verification, again separated by role. The result is more traits, but each trait states an actual capability. A plain argument cannot accidentally call `preprocess()`, while a preprocessed argument exposes that capability explicitly. Likewise, a verifier-only body does not need to implement or depend on the corresponding prover algorithm.

The second design requirement is to avoid duplicating machinery across the eight cases. The trait tree lets common structure be factored once: protocol identity, relation shape, channel execution, non-interactive proof vocabulary, preprocessing key derivation, and security metadata. Where a degeneration is useful, Argus provides it explicitly through adapters. For example, a plain protocol can be lifted into the preprocessing world by adding a trivial index and trivial keys, but this is an adapter rather than the core meaning of a plain argument.

3 The Channel Interface

The channel traits are the small boundary between protocol code and execution machinery. On the prover side, the protocol can only send prover messages and read verifier messages. On the verifier side, it can only read prover messages and send verifier messages. There is no sponge object in the protocol code, no explicit `absorb` call, no manual Fiat–Shamir challenge derivation, and no proof-byte parser.

This gives three useful interpretations of the same code. With the live backend, the verifier samples actual public coins and sends them over Rust channels. With `spongefish-dsfs`, prover messages are appended to the non-interactive argument string and absorbed into the sponge; verifier messages are squeezed from the transcript; and verification replays the same sequence from proof bytes. In the future, an iBCS compiler could produce exactly this kind of channel program: it would translate an IOP plus commitment machinery into an IA expressed through the same message interface.

The `spongefish-dsfs` crate is therefore part of the contribution. It is not just an external library call at the edge of the project. It implements the DSFS channel backend for the protocols written against Argus: prover channels, verifier channels, proof serialization, public-input binding, deterministic replay, and EOF checking. The transcript invariants are concentrated there: all public inputs are fixed before the first challenge, every prover message is absorbed before the next challenge is squeezed, verifier replay is deterministic, and malformed proof strings are rejected rather than accepted as prefixes.

This separation also clarifies what Argus does not claim. Writing a protocol against channels is not a security proof. It only removes one common source of implementation error: protocol authors no longer decide when to absorb messages, when to squeeze challenges, or how to replay proof bytes. Those choices are centralized in the backend. The actual security still comes from the protocol analysis, the DSFS theorem being applied, and the correctness of the backend implementation.

4 Role Separation and Asymmetric Compilation

A complete protocol specification normally contains both algorithms: the prover describes how to produce each message, and the verifier describes how to check the resulting conversation. This makes a unified authoring interface natural. It does not follow, however, that every consumer should depend on both algorithms. In recursion and accumulation, an outer prover may execute an inner verifier inside another protocol or circuit. That outer prover should be able to depend on the inner verifier without also importing the inner prover.

Argus separates these concerns by factoring each executable leaf trait into a prover half and a verifier half. For an interactive argument, the relationship is

`InteractiveArgument = InteractiveArgumentProver + InteractiveArgumentVerifier.`

The full trait is a method-free marker implemented automatically by every type that implements both halves. The same construction is applied uniformly to arguments and reductions, interactive and non-interactive execution, and plain and preprocessed protocols. Preprocessing itself remains on the shared preprocessing core because the indexer is neither a prover nor a verifier.

This finer trait surface does not make ordinary protocol authoring role-by-role. The authoring macros still accept one block containing protocol identity, relation types, `prove`, and `verify`; they generate the two capability implementations. Thus authoring unity and dependency separation are independent design choices: the source definition can present one conversation while generic consumers request only the role they need.

DSFS compilation follows the capabilities of its input:

$$\begin{aligned} IA_P &\xrightarrow{DSFS} NIA_P, \\ IA_V &\xrightarrow{DSFS} NIA_V, \\ IA_P + IA_V &\xrightarrow{DSFS} NIA_P + NIA_V. \end{aligned}$$

There is one compiler constructor rather than separate prover and verifier compilers. The wrapper gains a non-interactive proving implementation when its body implements the interactive prover half, and a verification implementation when its body implements the verifier half. A complete body therefore still compiles to a complete non-interactive argument, while a native one-sided body compiles without requiring the opposite algorithm.

Argus also provides role views of complete bodies. Calling `into_prover()` or `into_verifier()` is useful for party wiring and for preventing the wrong method from being called through a particular API. These views are not dependency boundaries: the wrapped type still contains a complete protocol implementation. Recursion requires a native verifier-only body, for which the prover implementation is absent from the type rather than merely hidden.

The two halves can be recombined when a complete object is required. `CombinedIA` joins compatible interactive bodies before compilation, while `CombinedNarg` joins independently compiled non-interactive halves. Associated types enforce agreement on the protocol shape, and construction checks the protocol identifiers in debug builds to catch accidentally mismatched halves. This refactor changes trait bounds and forwarding only: transcript state remains in the backend, and the `absorb` and `squeeze` sequence is unchanged.

5 State and Domain Separation

A central design question was whether protocol execution should be completely static. In the most austere design, `prove` and `verify` would take all parameters as explicit inputs: instance, witness, prover key, verifier key, or session. The protocol object itself would carry no meaningful configuration. This style is attractive because it makes hidden state hard to miss.

In practice, complex protocols often have configuration that is neither a per-claim instance nor a witness. WHIR-style and FRI-style protocols may carry rates, folding schedules, query counts, code information, hash choices, and other static protocol parameters. WARP has dimensions, code parameters, configuration, and relation descriptions. Keeping such information in `&self` is convenient and often the most natural Rust representation. Argus therefore allows protocol objects to be stateful in this immutable configuration sense.

This creates a hard domain-separation rule. Any data in `self` that affects the message flow, verifier challenges, accepted statements, proof layout, soundness profile, or transcript semantics must be reflected in `protocol_id(&self)` or otherwise absorbed as public input before the first

challenge. The method is intentionally a method on `&self`, rather than only an associated constant, so configured protocol objects can identify the actual compiled protocol they represent.

This distinction is important. It is acceptable for `self` to say which protocol is being run: for example, which code rate, folding schedule, or hash configuration is part of the protocol. It is not acceptable for `self` to secretly run the transcript: it should not contain a mutable sponge, Fiat–Shamir state, or hidden challenge counter. The backend channel owns that state. A useful review rule is: if changing `self` would make the verifier accept a different language, derive different challenges, or interpret proof bytes differently, then the domain separator must change too.

6 Preprocessing as Keys-as-Inputs

Preprocessing introduced a second statefulness question. Early designs stored preprocessing keys inside a prepared protocol object. Proving then looked like `prove(&self, instance, witness)`, with `self.pk` used internally. This was ergonomic, but it blurred the distinction between the protocol description and party-specific execution material.

Argus instead uses keys as explicit inputs: `preprocess(index)` returns a prover key and verifier key. At the interactive level, the body is run as `body.prove(ch, pk, instance, witness)` and `body.verify(ch, vk, instance)`. At the non-interactive level, DSFS hides the channel and adds a session and proof object: `pnia.prove(pk, session, instance, witness)` and `pnia.verify(vk, session, instance, proof)`. In both cases, the protocol body is not a prepared object containing a key.

This is cleaner because the prover and verifier naturally hold different keys, and because the same compiled wrapper can be reused for many independently preprocessed indexes. The committed-index binding is also explicit: DSFS derives public input bytes from `pk.committed_index()` on the prover side and `vk.committed_index()` on the verifier side, then absorbs those bytes together with the per-claim instance before the first challenge.

The prover/verifier trait split makes prepared key-carrying wrappers unnecessary. A preprocessing body can be compiled with only its prover or verifier capability, while the corresponding key remains an explicit argument to `prove` or `verify`. Capability separation and key possession are therefore represented independently: the type determines which algorithm is available, and the function argument determines which key is supplied.

7 Composition

Reductions are first-class because many proof systems do not end each interactive phase with accept/reject. A reduction transforms a source relation into a target relation. Sequential composition is therefore fundamental:

$$R_0 \xrightarrow{IR_1} R_1 \xrightarrow{IR_2} R_2.$$

Argus implements this with binary composition. Two reductions compose when the target instance and witness of the first match the source instance and witness of the second. Repeating the binary constructor gives arbitrary finite sequential chains, limited in practice by Rust type ergonomics and trait solving rather than by the abstraction.

A reduction followed by an argument is itself an argument for the original source relation. Argus represents this with a reduced-argument adapter. The prover first runs the reduction, obtaining a target instance and witness, then runs the final argument. The verifier computes the target instance by verifying the reduction and passes that target instance to the final argument verifier.

Preprocessing composition is especially informative. Composition must combine not only protocol execution but also indexes, prover keys, verifier keys, and committed-index bytes. Argus represents composed indexes and keys structurally, routes each sub-key to the corresponding component, and relies on the backend to bind the combined committed index before the first challenge. For example, if a preprocessing reduction is followed by a preprocessing decider, the index, prover key, and verifier key are pairs. The composed prover routes the first prover key to the reduction and the second to the decider; the verifier routes the verifier keys analogously. If keys had been stored inside prepared protocol objects, this composition would require assembling hidden-state objects. With keys as inputs, it is just typed routing.

Composition preserves role separation. A composed prover implementation depends only on the prover halves of its components, and the composed verifier depends only on their verifier halves. This requires separate prover-side and verifier-side forwarding implementations for each combinator. The additional boilerplate is a deliberate cost: a single combined implementation would reintroduce the dependency that the split was designed to remove.

The current composition framework is intentionally sequential. It does not try to be a general protocol DAG language. That restriction keeps the API small and matches the main use cases: chains of reductions and a final decider or argument.

8 Results

The project produced an executable interface layer with DSFS and live backends, examples, and case studies. Schnorr, sumcheck-style, and Bulletproofs protocols show the basic authoring model: the same channel code can compile through DSFS or run live.

All eight executable protocol forms have separate prover and verifier capabilities, and DSFS compilation preserves whichever capabilities are present on the input body. Tests cover complete bodies, role views, native one-sided bodies, and recombination before and after compilation. Proofs produced through the asymmetric paths are byte-identical to the complete compile path, **and the sigma-proofs SHAKE128 golden vectors continue to pass byte-for-byte**. These checks provide evidence that the change is type-level and does not alter transcript semantics.

The largest case study is WARP, a linear-time accumulation scheme for R1CS, modeled as a preprocessing interactive reduction plus a final preprocessing interactive argument. Static material such as configuration, code information, Merkle parameters, and relation data belongs to the index and is represented by committed-index bytes; per-claim fresh instances and accumulators remain ordinary instances. The WARP protocol engine uses channels for messages and challenges and does not own a sponge.

Bulletproofs gives a contrasting algebraic case study [4]. Unlike WARP, it does not need oracle openings: each round sends group commitments and receives a field challenge, which fits the IA channel directly. The inner-product argument is implemented both as a direct logarithmic protocol and as a one-round reduction composed down to a size-one decider; the range proof then reduces to the same inner-product core. **These examples show that Argus composition can re-produce a recursive hand-written protocol, while remaining example-grade rather than production implementations.**

Argus also includes instance-aware security metadata. Security profiles can be derived from concrete instances or explicit instance-family bounds, which is important when field size, round count, degree bounds, code parameters, or relation dimensions affect soundness. Composition combines these profiles along the reduction chain.

The sigma bridge is a smaller compatibility experiment. It rewrites the non-interactive sigma-

proofs pipeline in the Argus style and shows that sigma protocols can be driven through the same IA-to-DSFS interface. The existing sigma-proofs Fiat–Shamir path is already used by other consumers, so the bridge remains compatibility-oriented evidence rather than the main contribution.

9 Limitations and Future Work

Argus is a research prototype and has not received production cryptographic review. The interface enforces important implementation discipline, but transcript discipline is only one part of proof-system security.

The most important missing piece is the IOP-to-IA compiler. The interface was designed so that an iBCS-style compiler can output an IA, but the current workspace does not yet implement that compiler. This is the natural next step for connecting Argus to the broader arkworks-pq direction.

The prover/verifier split supplies a prerequisite for that downstream work, but its principal benefit has not yet been exercised by an actual recursive or iBCS consumer in this workspace. Role views are sufficient for deployment wiring and API restriction, but they do not remove the opposite algorithm from the dependency graph. Future integration should validate native one-sided bodies in the intended setting: an outer prover that depends on an inner verifier. That experience may also suggest better authoring support for one-sided implementations and less repetitive role-preserving composition.

The codec boundary could also be improved. The current channel layer is close to spongefish’s encoding and NARG serialization vocabulary. It remains open whether `NargSerialize` belongs on an abstract interactive channel trait, whether verifier reads should use `NargDeserialize` or the current `Deserialize` abstraction, and whether `ia-core` should own smaller codec traits. A related question is the sponge alphabet: channels already abstract the alphabet as a `Unit`, but the DSFS compiler, proof artifact, and sponge-parameter machinery remain byte-oriented. Supporting field-element transcripts should therefore be a backend extension, not a change to the message-flow abstraction.

Another open point is public-coin branching. A verifier may inspect a prover message before choosing the next challenge type, as long as that choice is a deterministic public function of the instance and transcript so far. It would not fit Argus if the challenge depended on hidden verifier state or manual transcript operations.

The WARP port illustrates why the IOP-to-IA compiler is the natural missing piece: its oracle commitments and openings are currently handled inside WARP with ordinary IA/IR messages, rather than generated by a general iBCS layer. Some WARP security terms are intentionally conservative and should be tightened against the exact analysis. Composition could also become more ergonomic: deeply nested chains and preprocessing tuples can become clumsy.

Finally, the statefulness rule deserves continued review. Configuration in `self` is necessary for realistic protocols, but every behavior-affecting parameter must be reflected in domain separation. This should remain a review checklist item for future WHIR, STIR, FRI, and STARK components.

10 Conclusion

Argus supplies a missing protocol layer in the arkworks proof-system stack. It lets authors describe arguments and reductions as typed public-coin channel programs, separates those descriptions from transcript mechanics, and provides DSFS and live backends for executing the same protocol objects. The design covers argument/reduction, interactive/non-interactive, and plain/preprocessed protocols through a bottom-up capability tree rather than a single maximal type. Within every

executable form, prover and verifier algorithms are separate capabilities: complete protocols remain convenient to author, while one-sided types and asymmetric DSFS compilation allow consumers to depend on only the role they require.

The main lesson is that IA/IR is an interface in its own right: an input to DSFS, a target for future iBCS compilation, and a protocol that can run interactively. Once that layer exists, transcript discipline, preprocessing, composition, role separation, and security metadata become backend and type-system concerns rather than ad hoc code inside each protocol.

References

- [1] Alessandro Chiesa and Michele Orru. *A Fiat-Shamir Transformation From Duplex Sponges*. IACR ePrint 2025/536, 2025. <https://eprint.iacr.org/2025/536>
- [2] arkworks-rs spongefish. Schnorr example. <https://github.com/arkworks-rs/spongefish/blob/main/spongefish/examples/schnorr.rs>
- [3] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. *Linear-Time Accumulation Schemes*. IACR ePrint 2025/753, 2025. <https://eprint.iacr.org/2025/753>
- [4] Benedikt Bünz et al. *Bulletproofs*. IEEE S&P, 2018.